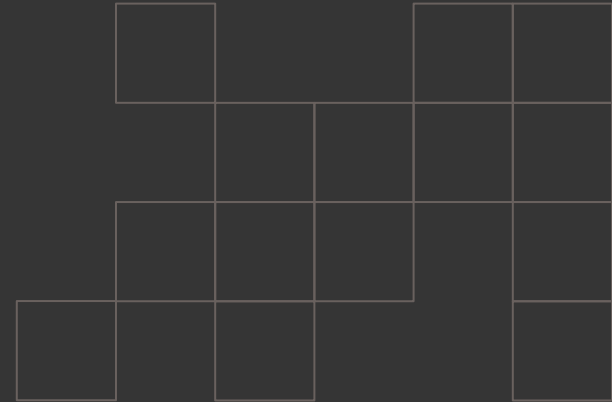




Project kickoff presentation



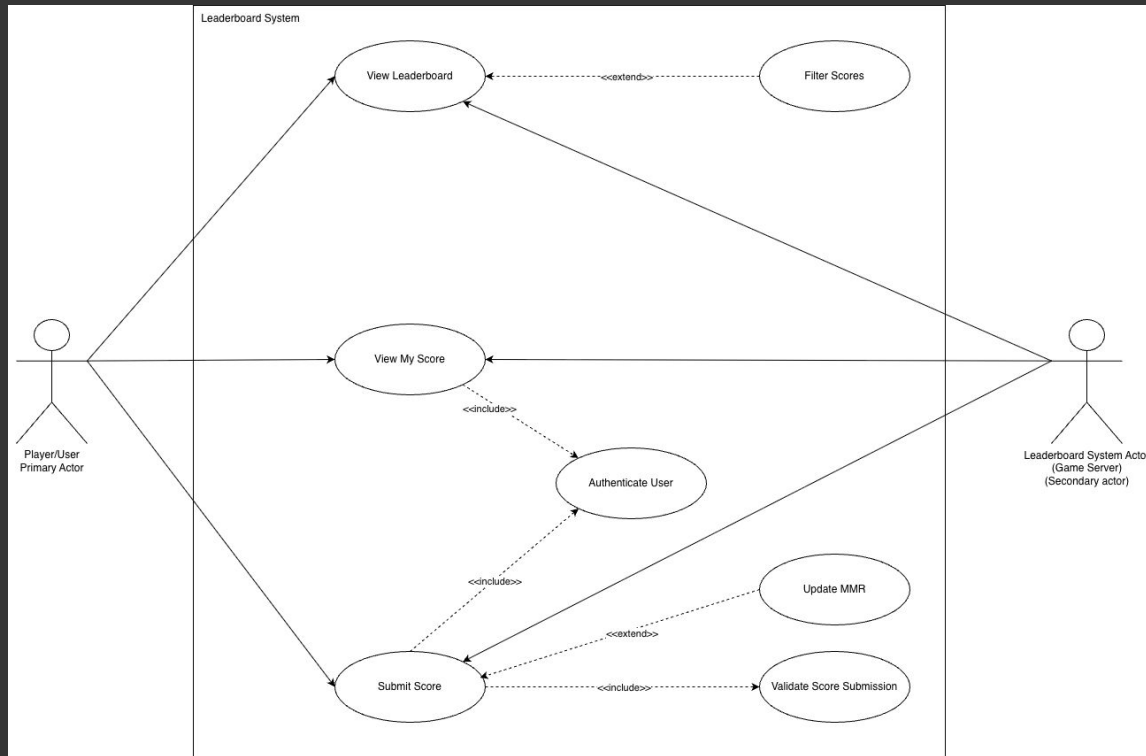
Contents

1.
Use Case Diagram
2.
Class Diagram
3.
State Machine Diagram
4.
Integration
5.
Challenges Identifies
6.
Design Critique
- 7
Moving Forward

USE CASE DIAGRAM

User and Server Use Cases:

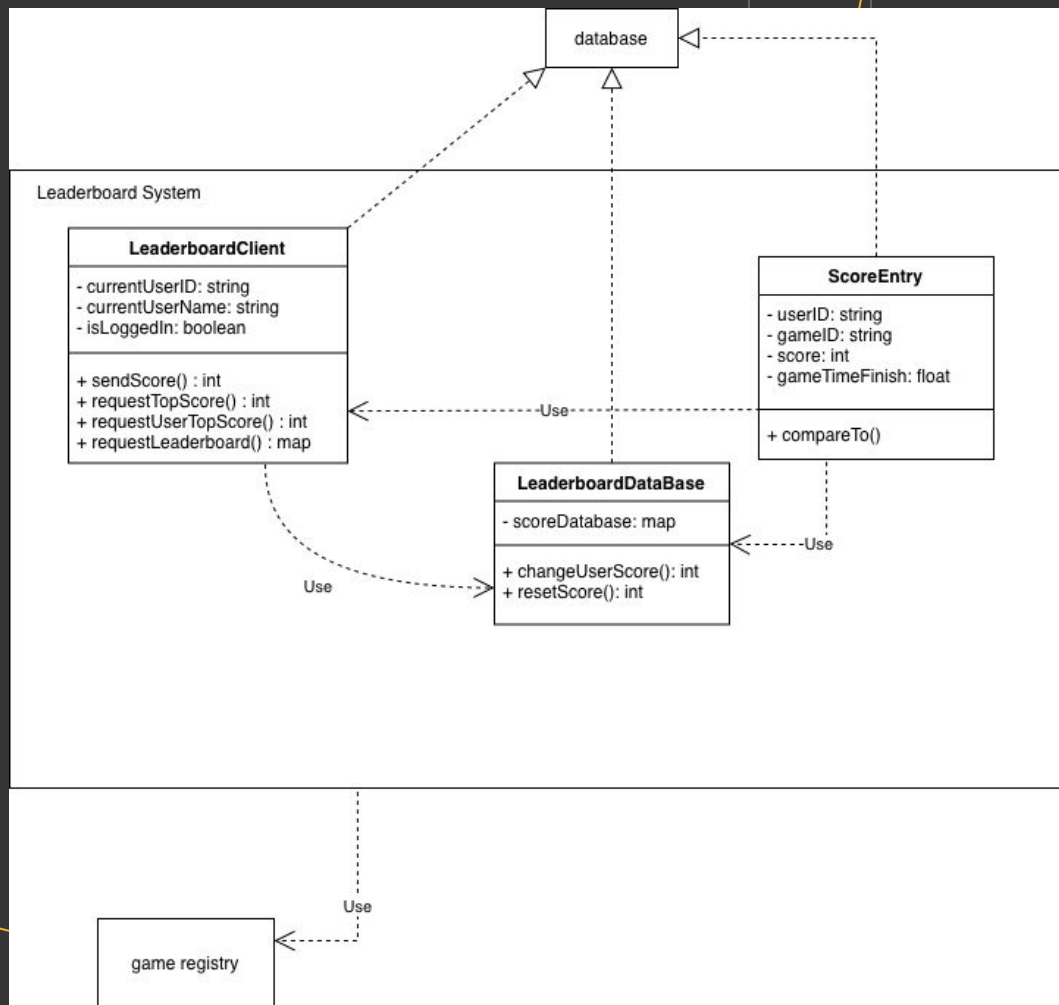
- View Leaderboard
- View My Score
- Submit Score
- Authenticate User
- Validate Score Submission
- Filter Scores
- Update MMR



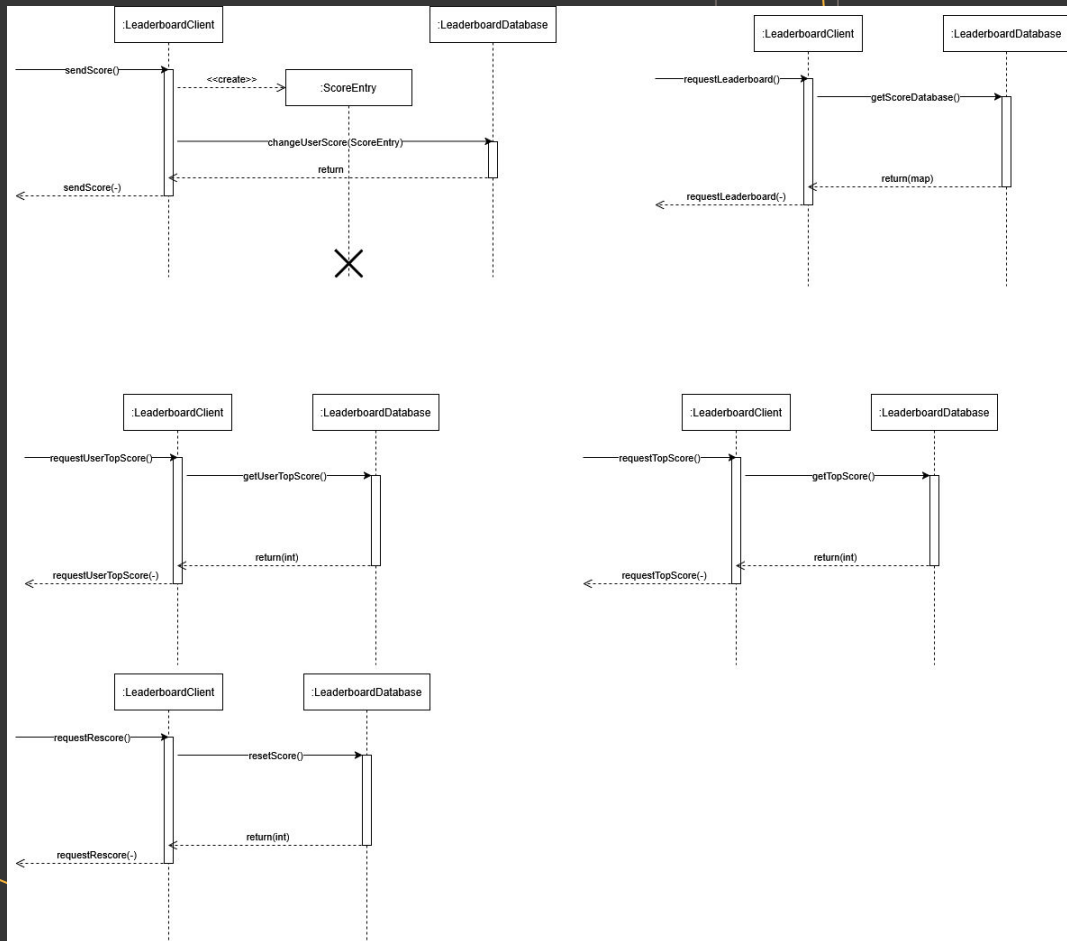
CLASS DIAGRAM

Classes and External Dependencies

- LeaderboardClient
- LeaderboardDatabase
- ScoreEntry
- Database (*external dependency*)
- Game Registry (*external dependency*)



SEQUENCE DIAGRAM



STATE MACHINE DIAGRAM

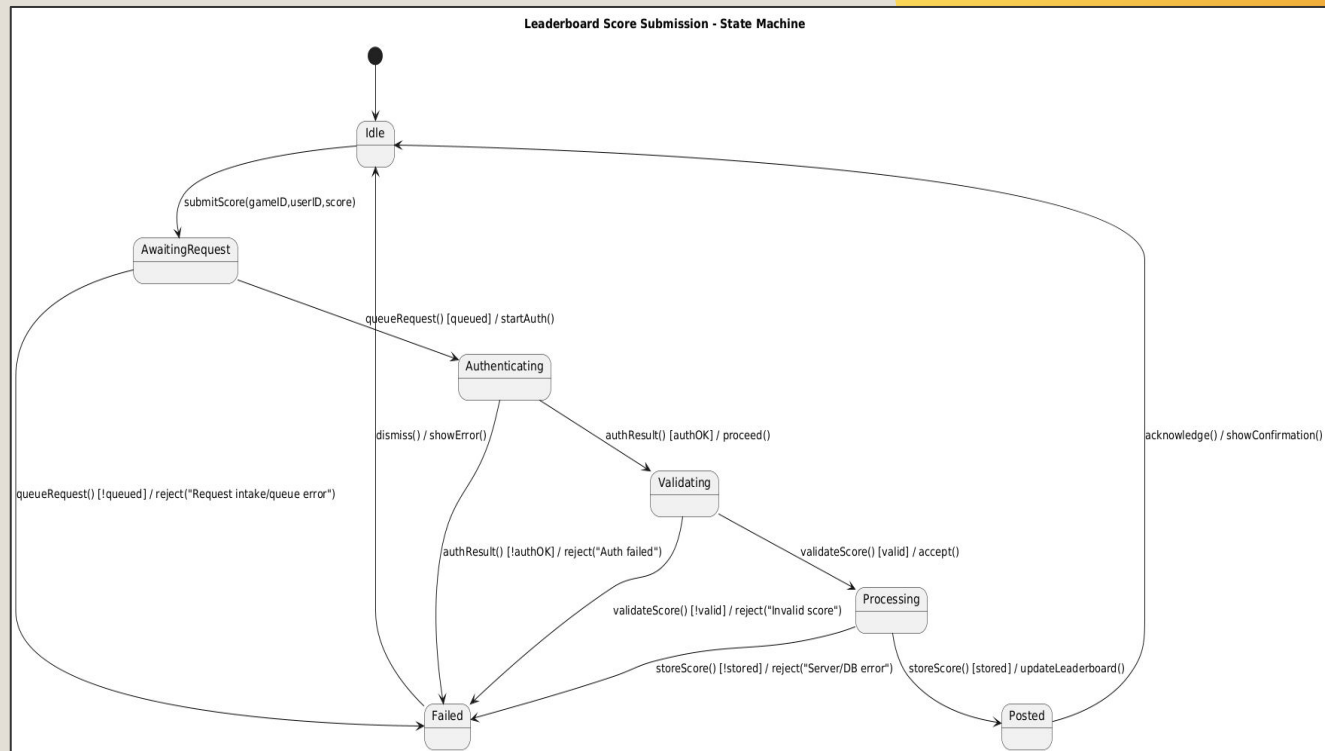
SCORE SUBMISSION LIFECYCLE

This diagram models the lifecycle of a score submission from initiation to completion or failure.

- Idle
- Authenticating
- Validating
- Processing
- Posted
- Failed

Separating authentication and validation ensures identity verification and rule compliance remain distinct responsibilities

Any failure transitions to a Failed state, ensuring no invalid data reaches persistence.



DESIGN RATIONALE

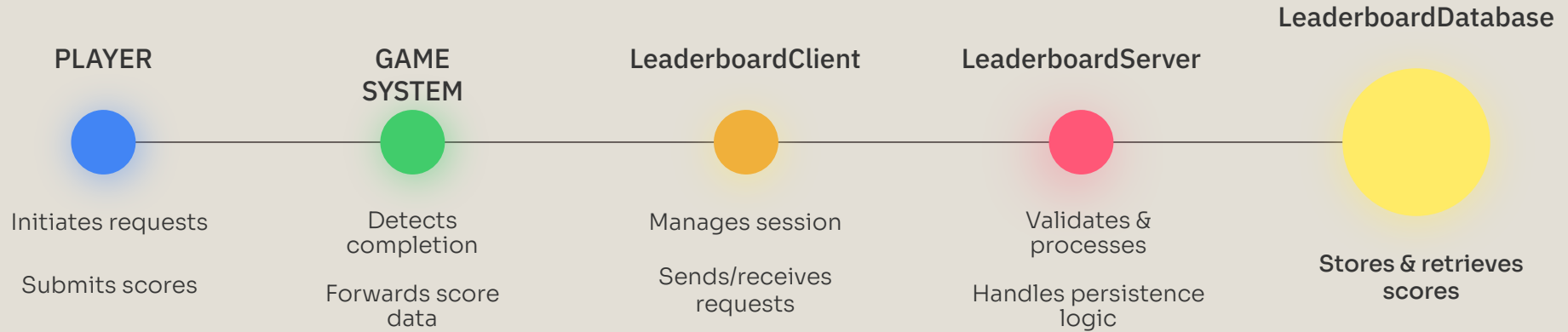
Server-side validation → Prevents client-side tampering and ensures all score submissions are verified before reaching persistence.

Layered architecture → Separates client, server, and database responsibilities to improve modularity, scalability, and maintainability.

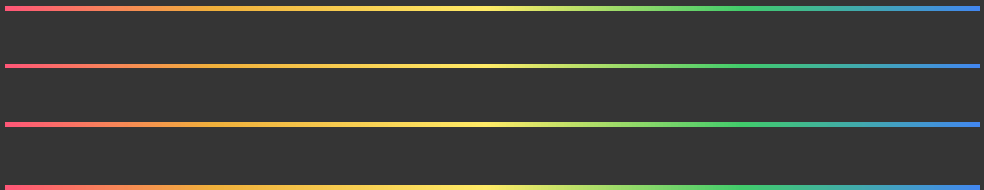
Separation of concerns → Authentication, validation, processing, and storage are handled in distinct components to reduce coupling and improve clarity.

Controlled failure states → All invalid or error conditions transition to a defined “Failed” state, ensuring system integrity and preventing corrupted data.

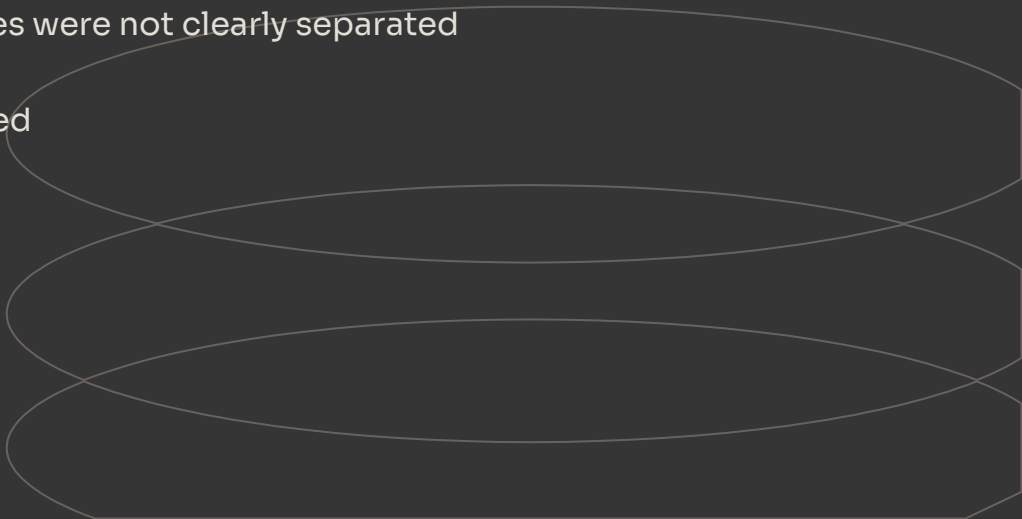
INTEGRATION



DESIGN CRITIQUE

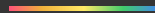
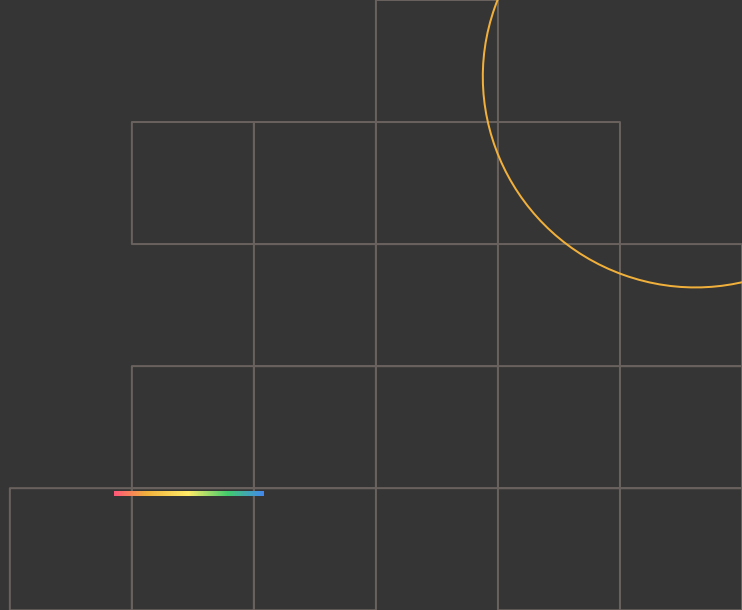


- Starter documentation lacked explicit failure modeling
- Authentication and validation responsibilities were not clearly separated
- Client-server boundaries were loosely defined
- Submission lifecycle was not fully modeled
- Persistence assumptions were ambiguous



CHALLENGES IDENTIFIED

- Interpreting unclear starter design assumptions
- Establishing strict responsibility boundaries between system layers
- Modeling safe and deterministic failure transitions
- Aligning integration contracts with other sub-teams
- Managing design depth within iteration constraints



Moving Forward

Step 1

Finalize subsystem interfaces and integration points

Step 2

Implement server-side validation and processing logic

Step 3

Complete database storage and retrieval mechanisms

Step 4

Perform end-to-end integration and testing

Step 5

Optimize for performance and scalability

Goal

Deliver a secure, modular, and production-ready leaderboard subsystem



Thank you

